

Practica 3. Localización: Filtro de partículas**9,5****1. Lee el maldito manual (0,5 puntos)****Pregunta #1: ¿Qué métodos de maps.py pueden ser útiles para completar el código solicitado en la Sección 3? ¿Para que los utilizaría?**

Para saber cuáles son los bordes del mapa nos va a venir bien utilizar el método `.bounds()` para definir los números entre los que se van a buscar las coordenadas aleatorias para acelerar la creación de partículas factibles (dentro del mapa). Además, para ver si los puntos aleatorios son válidos utilizaremos el método `.contains((x,y))`, y en el caso en el que este método devuelva `False` (partícula fuera del mapa), buscaremos otras coordenadas.

Pregunta #2: ¿Qué optimizaciones se han introducido en la clase Map para reducir los tiempos de ejecución de las Secciones 4 y 5?

En ambas secciones utilizamos el método `.check_collision(segment, compute_distance)`, el cual determina si un segmento interseca con las paredes del mapa y devuelve el punto de intersección y la distancia (si `compute_distance = True`). En la sección 4 lo utilizamos para corregir el movimiento de las partículas cuando atraviesan una pared durante la fase de predicción. Por otro lado, en la sección 5 lo utilizamos para simular las medidas del LiDAR en la fase de percepción calculando la distancia al obstáculo más cercano.

La clase Map incorpora algunas optimizaciones para reducir el tiempo de ejecución en las operaciones de colisión utilizadas durante las fases de **predicción (Sección 4)** y **percepción (Sección 5)** del filtro de partículas:

1. **Utiliza una librería compilada en C (libintersect):** El método `check_collision` puede utilizar una implementación compilada en C para calcular intersecciones entre segmentos (`segment_intersect`). Esto es mucho más rápido que la versión en Python, que tiene mayor coste computacional.
2. **Divide del mapa en regiones (use_regions):** El mapa se divide en celdas de aproximadamente 1×1 m, y para cada región se precomputan los segmentos del mapa potencialmente visibles. Por lo tanto, cuando se calcula una colisión solo se comprueban los segmentos cercanos a la posición del robot o la partícula, en lugar de todos los segmentos del mapa.
3. **Filtra por rango del sensor:** En `_init_regions` se descartan los segmentos que quedan fuera del rango del LiDAR. Esto reduce todavía más el número de comprobaciones necesarias durante la simulación de los rayos.

Gracias a estas optimizaciones se reduce significativamente el número de cálculos de intersección necesarios, acelerando las fases de movimiento de partículas y simulación de medidas del LiDAR, que son las más costosas del algoritmo.

2. ¿Perdido o encontrado? (1 punto)

TODO: 3.1. Create the /pose publisher (PoseStamped message).

```
self._pose_publisher = self.create_publisher(
    msg_type=PoseStamped,
    topic="pose",
    qos_profile=10
)
```

TODO: 3.2. Complete the function body with your code (i.e., replace the pass statement).

```
new_msg = PoseStamped()
new_msg.header.stamp = self.get_clock().now().to_msg()
new_msg.localized = self._localized
if self._localized:
    new_msg.pose.position.x = x_h
    new_msg.pose.position.y = y_h
    w,x,y,z = euler2quat(0, 0, theta_h)
    new_msg.pose.orientation.w = w
    new_msg.pose.orientation.x = x
    new_msg.pose.orientation.y = y
    new_msg.pose.orientation.z = z
self._pose_publisher.publish(new_msg)
```

TODO: 3.3. Sync the /pose and /cmd_vel subscribers if enable_localization is True.

```
if enable_localization:
    self._subscribers.append(
        message_filters.Subscriber(
            self,
            PoseStamped,
            "pose",
            qos_profile=10
        )
    )

ts = message_filters.ApproximateTimeSynchronizer(
    self._subscribers,
    queue_size=10,
    slop=9
)

ts.registerCallback(self._next_step_callback)
```

3. No tengo ni idea de dónde estoy (1punto)

```
particles = np.empty((particle_count, 3), dtype=object)
```

```

# TODO: 3.4. Complete the missing function body with your code.
xmin, ymin, xmax, ymax = self._map.bounds()
valid_thetas = [0, math.pi / 2, math.pi, 3 * math.pi / 2]
if global_localization:
    for i in range(particle_count):
        x = np.random.uniform(xmin, xmax)
        y = np.random.uniform(ymin, ymax)

        while not self._map.contains((x, y)):
            x = np.random.uniform(xmin, xmax)
            y = np.random.uniform(ymin, ymax)

        theta = valid_thetas[np.random.randint(0, 4)]
        particles[i] = (x, y, theta)
else:
    for i in range(particle_count):
        x = np.random.normal(initial_pose[0], initial_pose_sigma[0])
        y = np.random.normal(initial_pose[1], initial_pose_sigma[1])

        while not self._map.contains((x, y)):
            x = np.random.normal(initial_pose[0], initial_pose_sigma[0])
            y = np.random.normal(initial_pose[1], initial_pose_sigma[1])

        theta = np.random.normal(initial_pose[2], initial_pose_sigma[2])
        particles[i] = (x, y, theta % (2 * math.pi))

return particles

```

4. Las partículas salen de excursión... y se chocan (1punto)

```

# TODO: 3.5. Complete the function body with your code.
for i in range(len(self._particles)):
    v_new = np.random.normal(v, self._sigma_v)
    w_new = np.random.normal(w, self._sigma_w)
    x_last = self._particles[i][0]
    y_last = self._particles[i][1]
    x = self._particles[i][0] + v_new * math.cos(self._particles[i][2]) *
self._dt
    y = self._particles[i][1] + v_new * math.sin(self._particles[i][2]) *
self._dt
    theta = (self._particles[i][2] + w_new * self._dt) % (2 * math.pi)
    intersection, _ = self._map.check_collision([(x_last, y_last), (x,
y)], False)
    if intersection:
        x, y = intersection
        self._particles[i][0] = x

```

Esto es poco pythónico. ¿Por qué no usar for particle in self._particles?

```
self._particles[i][1] = y
self._particles[i][2] = theta
```

5. Echando un vistazo virtual alrededor (1 punto)

```
z_hat: list[float] = []

# TODO: 3.6. Complete the missing function body with your code.
rays = self._lidar_rays(pose, indices=np.arange(8).tolist(),
degree_increment=45)

for segment in rays: Las variables tienen que estar en inglés.
    _, distancia = self._map.check_collision(segment, True)
    z_hat.append(distancia)
return z_hat
```

6. Estoy de acuerdo en estar en desacuerdo (0,5 puntos)

```
# TODO: 3.7. Complete the function body (i.e., replace the code below).
return 1 / (sigma * math.sqrt(2 * math.pi)) * math.exp(-((x - mu) ** 2 /
(2 * sigma**2)))
```

Si dividís al final os ahorrarás una operación.

7. ¿Crees lo que ven tus ojos? (1 punto)

```
probability = 1.0

# TODO: 3.8. Complete the missing function body with your code.
measurements = [
    measure if not math.isnan(measure) else self._sensor_range_min
    for measure in measurements
]
z_hat = self._sense(pose=particle)
z_hat = [z if not math.isnan(z) else self._sensor_range_max for z in
z_hat]
for z, measurement in zip(z_hat, measurements[::len(measurements)//8]):
    probability *= self._gaussian(mu=z, sigma=self._sigma_z,
x=measurement)
return probability
```

¡No tiene sentido que reemplacéis unas medidas por el mínimo y otras por el máximo!

8. Solo sobreviven los más aptos (1 punto)

```
# TODO: 3.9. Complete the function body with your code (i.e., replace the
pass statement).

weights = np.array(
    [self._measurement_probability(measurements, p) for p in
self._particles]
```

```

)
w_sum = weights.sum()
# normalizar y construir CDF ¡En inglés!
weights /= w_sum
cdf = np.cumsum(weights)

# Muestreo
step = 1.0 / self._particle_count
u0 = np.random.uniform(0.0, step)
u = u0 + step * np.arange(self._particle_count)

# mapear punteros u a índices vía CDF
idx = np.digitize(u, cdf, right=False)

# construir nuevo conjunto de partículas
self._particles = np.array([self._particles[i] for i in idx])
self.particles[idx] funciona.

```

9. La unión hace la fuerza (1 punto)

```

# TODO: 3.10. Complete the missing function body with your code.
localized: bool = False
pose: tuple[float, float, float] = (float("inf"), float("inf"),
float("inf"))
particles_sincos = np.array(
    [[particle[0], particle[1], math.sin(particle[-1]),
math.cos(particle[-1])]
    for particle in self._particles]
)
clustering = DBSCAN(eps=0.2, min_samples=20).fit(particles_sincos)
n_clusters = len(set(clustering.labels_)) - (1 if -1 in
clustering.labels_ else 0)
min_number_particles = 200
max_number_particles = self._initial_particle_count
wanted_particles = (
    min_number_particles * n_clusters
    if min_number_particles * n_clusters < max_number_particles
    else max_number_particles
)
self._particle_count = wanted_particles

if n_clusters == 1:
    localized = True
    # self._particle_count = 50
    pose_sincos = (
        np.mean(particles_sincos[:, 0]),
        np.mean(particles_sincos[:, 1]),
        np.mean(particles_sincos[:, 2]),
        np.mean(particles_sincos[:, 3]),

```

```
)  
    pose = pose_sincos[0], pose_sincos[1], math.atan2(pose_sincos[-2],  
pose_sincos[-1])  
return localized, pose
```

10. Análisis de sensibilidad (0,5 puntos)

Me hubiera gustado que incluyerais algunos experimentos con resultados numéricos para apoyar la explicación.

1/2

Pregunta #3: ¿Cuál es el efecto de incrementar o disminuir el número de partículas a nivel de tiempos de computación y exactitud de la localización? ¿Cuál es el valor inicial más apropiado de este hiperparámetro para este entorno? ¿Qué estrategia de reducción de partículas funciona mejor?

El número de partículas afecta tanto al tiempo de computación como a la precisión de la localización. Si se incrementa el número de partículas, el filtro puede representar mejor la distribución de probabilidad de la pose del robot, lo que suele producir una estimación más precisa. Sin embargo, el coste computacional aumenta significativamente, ya que todas las partículas deben pasar por las fases de predicción, cálculo de medidas y remuestreo, lo que incrementa el tiempo de ejecución.

Por el contrario, si se reduce el número de partículas, el algoritmo se ejecuta más rápido, pero la representación de la creencia sobre la pose es peor. Esto puede provocar que converja más lento o incluso que estime incorrectamente si el número de partículas es demasiado bajo. En simulación, un valor inicial de alrededor de 2000 partículas nos resultó adecuado para resolver el problema de localización global: permitió localizarnos globalmente y no supuso un coste computacional elevado.

Una vez que el filtro converge y las partículas se agrupan en un único clúster, decidimos reducir progresivamente el número de partículas para bajar la carga computacional. Nuestra estrategia se basó en ajustar el número de partículas en función del número de clusters detectados, manteniendo solo un subconjunto reducido cuando queda un clúster. Esto permite mantener un seguimiento eficiente de la pose del robot sin necesidad de conservar un gran número de partículas.

1/2

Pregunta #4: ¿Qué sucede si modifica los parámetros de ruido de movimiento y medida?

Los parámetros de ruido del modelo de movimiento y del modelo de medida influyen directamente en el comportamiento y la convergencia del filtro de partículas.

Si se incrementa el ruido de movimiento (σ_w y σ_y), las partículas se dispersan más después de cada paso de predicción. Esto puede ser útil para evitar que el filtro quede atrapado en una hipótesis incorrecta, pero si el ruido es demasiado alto la estimación de la pose se vuelve muy inestable y el filtro tarda más en converger. Por el contrario, si el ruido de movimiento es demasiado bajo, las partículas tienden a concentrarse excesivamente y el filtro puede volverse demasiado confiado en su estimación. En este caso, si el robot se desvía de la trayectoria esperada debido a errores reales de movimiento, el filtro puede perder la localización.

De nuevo, habría estado bien incluir algunos experimentos.

En cuanto al ruido de medida, si su valor es bajo el filtro confía mucho en las medidas del sensor, lo que hace que las partículas que se desvíen un poco de las observaciones reciban pesos muy pequeños y no se remuestreen. Si el ruido de medida es demasiado alto, el filtro considera muchas más partículas como candidatas (probabilidad alta), lo que reduce la discriminación entre hipótesis y puede ralentizar la convergencia.

Robot real

Launch file:

Hemos cambiado y ejecutado el lab02.launch.py (ignorar el lab03.launch.py):

```
from launch import LaunchDescription
from launch_ros.actions import LifecycleNode, Node

import math

def generate_launch_description():
    # start = (1.0, -1.0, 0.5 * math.pi) # Outer corridor
    # start = (0.6, -0.6, 1.5 * math.pi) # Inner corridor

    simulation = False
    world = "lab03"
    start = (0.0, -0.8, math.radians(90))
    particles = 2000
    sigma_v = 0.05
    sigma_w = 0.1
    sigma_z = 0.2

    wall_follower_node = LifecycleNode(
        package="amr_control",
        executable="wall_follower",
        name="wall_follower",
        namespace="",
        output="screen",
        arguments=["--ros-args", "--log-level", "WARN"],
        parameters=[{"simulation": simulation}],
    )

    particle_filter_node = LifecycleNode(
        package="amr_localization",
        executable="particle_filter",
        name="particle_filter",
        namespace="",
        output="screen",
        arguments=["--ros-args", "--log-level", "WARN"],
        parameters=[
            {
```

```

        "enable_plot": True,
        "global_localization": True,
        "particles": particles,
        "sigma_v": sigma_v,
        "sigma_w": sigma_w,
        "sigma_z": sigma_z,
        "simulation": simulation,
        "world": world,
    }
],
)

odometry_node = Node(
    package = "amr_turtlebot3",
    executable = "odometry_node",
    arguments=["--ros-args", "--log-level", "WARN"],
)

lifecycle_manager_node = Node(
    package="amr_bringup",
    executable="lifecycle_manager",
    output="screen",
    arguments=["--ros-args", "--log-level", "WARN"],
    parameters=[
        {
            "node_startup_order": (
                "particle_filter",
                "wall_follower", # Must be started last
            )
        }
    ],
)

return LaunchDescription(
    [
        odometry_node,
        particle_filter_node,
        wall_follower_node,
        lifecycle_manager_node, # Must be launched last
    ]
)

```

11. Arre, so, arre, so (1,5 puntos)

1) WallFollowerExecution.msg:

std_msgs/Header header

bool `wallfollower` Este nombre no indica para qué sirve el campo.

También hemos añadido este tipo de mensaje en el archivo CMakeLists.txt

2) y 3) Cambios en `particle_filer_node.py`:

En el `on_configure()`:

```
self._wall_follower_publisher = self.create_publisher(
    msg_type=WallFollowerExecution,
    topic="wallfollower", De nuevo, este nombre es mejorable.
    qos_profile=10
)

# Subscribers
scan_qos_profile = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10,
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
)

self._z_vels = []
self._z_scan = [0.22] * 240
self._odom_subscriber = self.create_subscription(Odometry, "odometry",
self._process_odom_callback, qos_profile=scan_qos_profile)
self._scan_subscriber = self.create_subscription(LaserScan, "scan",
self._process_scan_callback, qos_profile=scan_qos_profile)
self._timer = self.create_timer(5, self._compute_pose_callback)
```

Nuevos callbacks:

```
def _process_odom_callback(self, msg: Odometry):
    z_v = msg.twist.twist.linear.x
    z_w = msg.twist.twist.angular.z
    if abs(z_v) > 0.01 or abs(z_w) > 0.08:
        self._z_vels.append((z_v, z_w))

def _process_scan_callback(self, msg: LaserScan):
    self._z_scan = msg.ranges
```

Cambios en `wall_follower_node.py`:

En el `on_configure()`:

```
self._can_follow_wall = True
self._wall_follower_subscriber = self.create_subscription(
    WallFollowerExecution, "wallfollower",
    self._process_wallfollower_callback, qos_profile=10
)
```

Nuevo callback:

```
def _process_wallfollower_callback(self, msg: WallFollowerExecution):
    self._can_follow_wall = msg.wallfollower
```

Por último, en `_compute_commands_callback()`, si no está localizado, añadimos la siguiente lógica:

```
if self._can_follow_wall:
    # Execute wall follower
    v, w = self._wall_follower.compute_commands(z_scan, z_v, z_w)
    self.get_logger().info(f"Commands: v = {v:.3f} m/s, w = {w:.3f} rad/s")
else:
    self.get_logger().info(f"SE HA PARADO.") En inglés.
    v = 0.0
    w = 0.0

# Publish
self._publish_velocity_commands(v, w)
```

4) Por último, implementamos el filtro de partículas cuando el robot está quieto (y no dejamos que se mueva hasta que acabe el filtro). Luego publicamos la pose estimada.

```
def _compute_pose_callback(self):
    """Subscriber callback. Executes a particle filter and publishes
    (x, y, theta) estimates.

    Args:
        odom_msg: Message containing odometry measurements.
        scan_msg: Message containing LiDAR sensor readings.

    """
    self.get_logger().info(f"SE HA LLAMADO AL WALLFOLLOWER MESSAGE.")
    new_msg = WallFollowerExecution()
    new_msg.wallfollower = False
    new_msg.header.stamp = self.get_clock().now().to_msg()
    self._wall_follower_publisher.publish(new_msg)

    for z_v, z_w in self._z_vels:
        # Execute particle filter
        self._execute_motion_step(z_v, z_w)
        self._steps += 1

    x_h, y_h, theta_h = self._execute_measurement_step(self._z_scan)
    self._z_vels = []

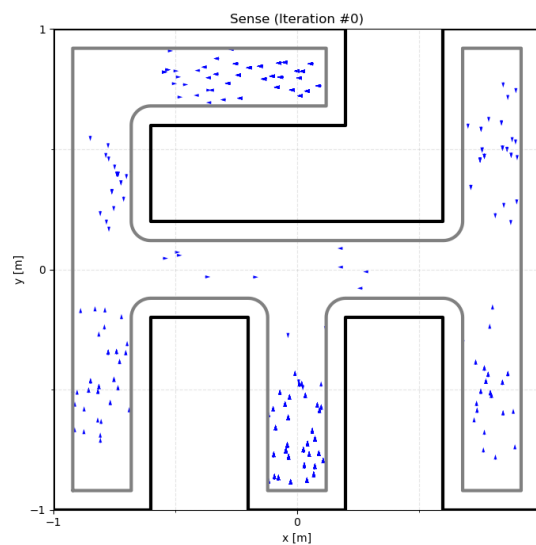
    new_msg = WallFollowerExecution()
```

```
new_msg.wallfollower = True
new_msg.header.stamp = self.get_clock().now().to_msg()
self._wall_follower_publisher.publish(new_msg)
# Publish
self._publish_pose_estimate(x_h, y_h, theta_h)

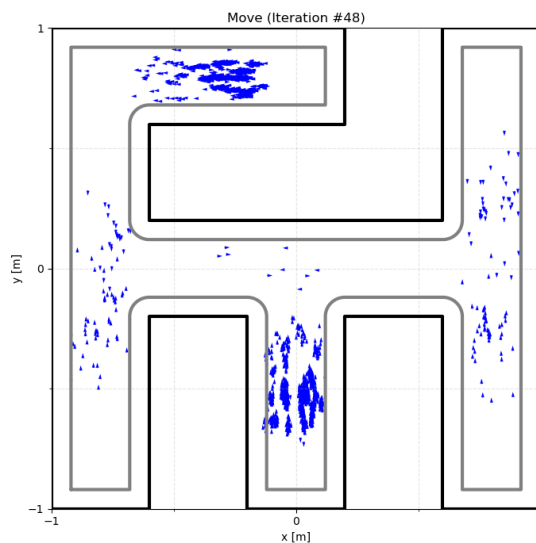
self._timer.reset()
```

Además, ejecutamos este callback manualmente en el `on_activate()` para hacer un primer sense antes de movernos, con el objetivo de eliminar partículas malas, tener menos clusters (y partículas), y por lo tanto bajar la carga computacional del algoritmo.

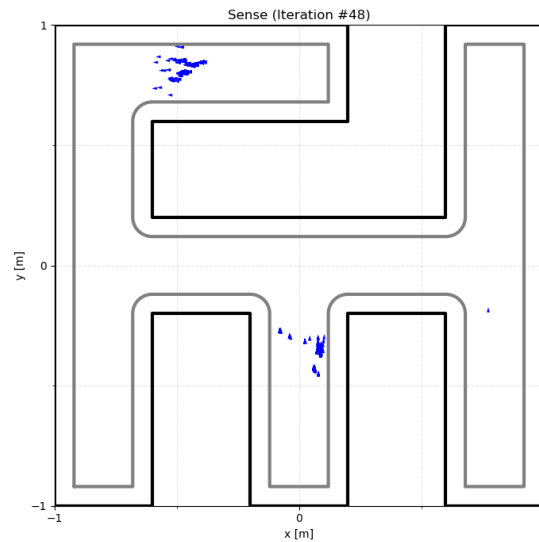
Resultados:



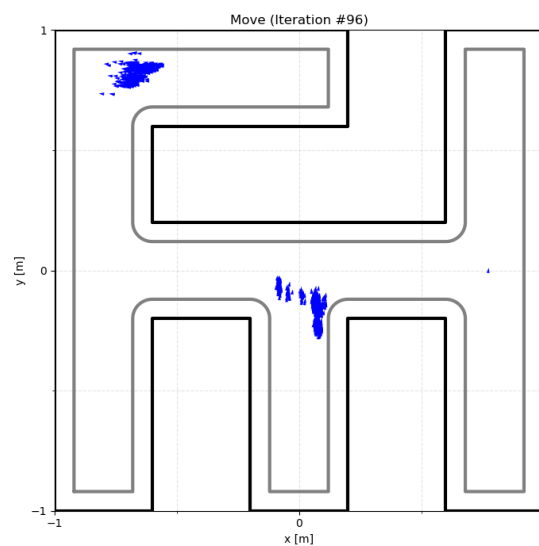
Primer “sense” descarta muchas partículas



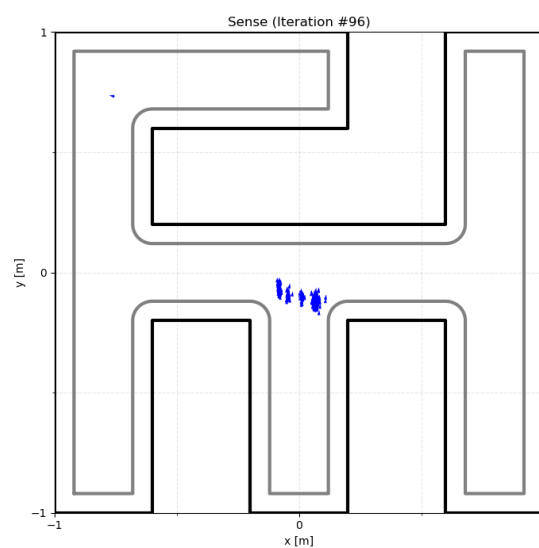
Ultimo “move” antes del segundo “sense”



Segundo “sense”



Ultimo “move” antes del tercer “sense”



Tercer y último “sense” (se ha localizado)